
Computer Systems: An Insight into their Build and the Evolution of Hardware

Rishabh S. Pomaje*
Pune, India, 411038.
rishpresearch@gmail.com
July 27, 2025

Abstract

Computer systems have become integral to modern engineering applications, comprising distinct software and hardware components, each organized into multiple layers of abstraction. The software stack typically includes the user interface, high-level programming constructs, and low-level executable code. It forms a mature field of engineering and is an extensively studied domain. At the same time, the hardware stack spans from microarchitectural design to logic gates, transistor-level circuits, and physical layout implementations. This article provides an overview of these hardware abstraction layers, establishing a foundation for understanding the systematic evolution of hardware design methodologies and technologies. For the latter, a popular idea known as **Moore's Law** will be considered for reference.

1 Introduction

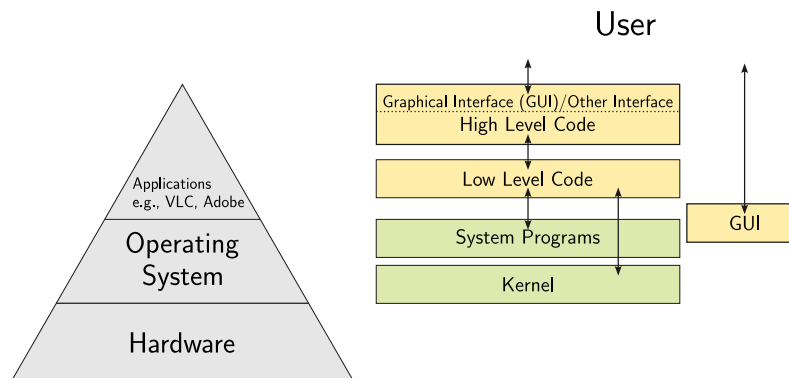


Figure 1: Layers of abstraction in a software system.

We all interact with computers on a daily basis. These devices have become ubiquitous, taking different shapes and form. We have palm-sized smartphones, notebook-sized laptops, and larger desktop personal-computers. Computers are also embedded in vending machines, self-service kiosks, and ATM machines.

**Disclaimer:* The content presented in these blog posts reflects my personal thoughts, and interpretations. The primary purpose of this blogs series is to improve my technical writing. I do not claim any sort of authoritative accuracy or completeness of the information provided. These posts are not intended for educational, commercial, or monetary gain of any kind. Readers are advised to use their own discretion when interpreting or applying the content.

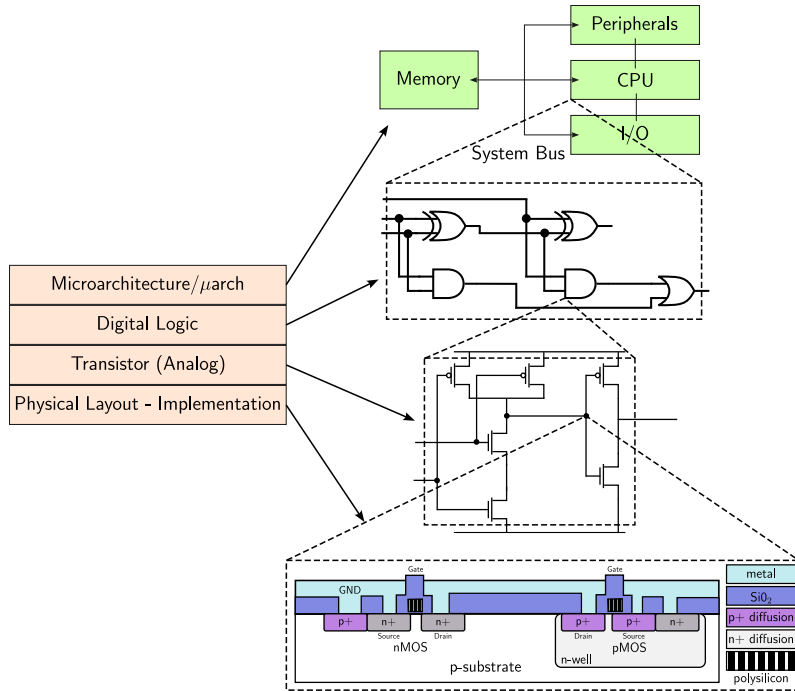


Figure 2: Layers of abstraction in the hardware part of an electronic system.

The inclusion of such devices is driven by their simple operational mechanism. This ease of interaction is enabled by the principle of **abstraction** alongside decades of expert knowledge and innovation in the field of electronics and computer science. Modern computer systems, with nice looking graphics, are built in layers of abstraction. Figure 1 and Figure 2 for a visually illustrate a breakdown of these layers².

In this article, we will focus on the hardware developments from a historical perspective as well as what the future might bring. Prior to this, we briefly introduce and discuss each abstraction layer.

2 Abstraction Principle

Abstraction allows individual subject experts to design, develop and implement efficient, robust, and functionally accurate complex systems. A top-down approach will help a novice better understand the layers as it starts with the most familiar and interacted part of a computer system, the software.

2.1 Software

Figure 1 describes the layers of abstraction involved in a software sub-system. The topmost layer is the **Graphical User Interface (GUI)** through which user interacts with the device. This provides a standardized system operating model. Thus, the user is “abstracted” from the technical working details underneath.

Beneath the GUI³ lies a well-structured software stack. The GUI and its functioning is designed by computer programmers using *high-level programming languages*. A **programming language**, in-general, is the medium of instruction between the human and the machine. However, every machine is different and it necessitates programs that are portable across different hardware platforms. This led to the rise of **Operating Systems (OS)** concept.

²Actual systems are far more complicated and may contain more details. The figures provide a simplified view.

³Nowadays, GUI referred as simply User Interface (UI).

2.2 Operating System

An operating system manages the hardware resources of a computer, schedules different processes, and provides an efficient environment for the user application programs to use the hardware. As with many complex systems, there is no complete definition of an OS. Generally, an OS is composed of three parts:

- Kernel: This is a core program that runs in the background.
- System Programs: Essential programs for basic functioning of the system.
- Non-System/User Programs: Applications that cater to the end user. Example, VLC media player, PDF Reader, etc.

2.3 Instruction Architecture

At a lower level, the OS manages the hardware through a formal specification known as the **Instruction Set Architecture (ISA)**. The ISA provides the fundamental programming model of the **Central Processing Unit (CPU)**. It defines the hardware resources such as the data types, registers, memory, and their access methods. The ISA includes mnemonics for the basic instructions the CPU is able to perform. Consider the following example:

ISA – Assembly Programming

Consider adding two numbers stored in registers named R1 and R2 and placing the result in register R3. The instruction for this would look like:

```
add R3, R1, R2 // Do R3 = R1 + R2
```

Additionally, if we want to move the sum to a memory location 0x2A, and load register R3 with contents of register R4,

```
str R3, [R0] // R0 holds 0x2A. Store R3 to location pointed by R0
mov R3, R4   // R4 = R3
```

Thus, assembly language (specified by the ISA) is considered the lowest level of language that both, the human and the computer⁴, can understand.

It is obvious that writing assembly code for even the simplest of tasks such as grade calculation is tedious. Thus, another program known as the **Compiler** is used to automate the process of converting high-level human-written code to machine-readable binaries. Modern compilers⁵ follow a multi-step process and their design is a field of study (Compiler Design) in itself. Since it is bridge between the hardware and the software, it is essential for a computer architect to be well-versed in the compiler design theory.

Depending on the complexity and capability of the ISA, systems are typically classified as either **Reduced Instruction Set Computer (RISC)** or **Complex Instruction Set Computer (CISC)**. RISC machines involve simple instructions and thus, complex softwares whereas CISC architectures incorporate more complex instructions directly in the hardware.

2.4 Hardware – Microarchitecture, Gates, and Transistors

This microarchitecture (also referred as μ arch/uarch) includes the hardware details at a *register-word*⁶ level. Microarchitecture is the way a given ISA is implemented in a particular processor. It is possible to have multiple implementations for the same ISA. For example, there are two main types of information: 1. The program (i.e., the instructions) and 2. The data. Depending on whether both kinds of information are stored in the same unified memory or different, we have **Von Neumann** and

⁴Technically, the computer can only understand binary symbols (represented by voltage levels). For each instruction, there is a corresponding binary code pattern, known as opcode, is pre-defined.

⁵Since compilers are essentially translators, they depend on the input language and possibly on the target hardware. If interested, you may look-up compiled languages versus interpreted languages and cross-compilers.

⁶The size of a word (in bits) depends on the bit-width of the Arithmetic Logic Unit (ALU).

Harvard architecture, respectively. Each of them have their own merits and their suitability depends on the requirements of the application.

Descending further, the microarchitecture is realized at the *gate-level*, composed of *logic-gates*, followed by the transistor-level circuits of each gate and finally, the actual physical fabrication of the transistors. Typically, modern digital systems are built using “MOS” which stands for Metal Oxide Semiconductor technology.

3 Computer Hardware Development

The single most impactful of predictions regarding the evolution of the electronics circuit, especially semiconductor devices, was made by *Dr. G. E. Moore* in his article [Moore \(1998\)](#). This vision, now famously known as **Moore’s law**, has shaped the trajectory of integrated electronics for decades. In this section, we try and understand his vision and explore how the semiconductor industry has evolved from a historical perspective.

3.1 Moore’s Law

[Moore \(1998\)](#) predicted the invention of home-computers, autonomous controls for vehicles, and personal, portable communication equipment. Looking back, not only have these predictions materialized, but we’ve often exceeded them. One particularly striking example is the modern smartwatch—an ultra-compact, powerful, always-connected computing device.

Rather than a physical or scientific one, Moore’s Law emphasizes more on the economics and user-value of electronics. Broadly, it suggests that the users get twice the value from the electronics every 18-24 months. However, several technological details and economical insights were provided in [Moore \(1998\)](#). Before understanding these, let us look at the merits of integrated electronics.

Through applications such as the *Apollo mission*, it was validated that integrated electronics were at least as reliable as discrete electronics. In addition, they had the added advantage of lesser weight and smaller size. With improving integration technology, it was expected that the products would demonstrate higher reliability, reduced costs, and most importantly, improved, higher performance.

3.2 Historical Analysis

To understand analyze the development of the semiconductor industry, we use a data⁷ consisting of 2,185 CPUs and 2,668 Graphical Processing Units (GPUs)⁸.

Moore’s Law primarily centers on increasing the number of transistors per chip, thereby enhancing functionality. This growth is enabled by advances such as transistor miniaturization and improved lithography. [Figure 3](#) reveals a sustained increase in transistor density over time, which correlates with shrinking process nodes, the minimum feature size of transistors, shown in [Figure 4](#).

Dennard Scaling, another key principle, suggested that as transistors shrink, power consumption should scale down proportionally, keeping power density constant over time. It implied scaling voltage and current with transistor length. However, practical limitations, such as noise susceptibility at lower voltage levels, caused Dennard Scaling to break down earlier than Moore’s Law.

One of the most critical parameters in modern chip design is power consumption, commonly measured in terms of **Thermal Design Power (TDP)**⁹. To quote a question from [Moore \(1998\)](#), “*Will it be possible to remove the heat generated by tens of thousands of components in a single chip?*”. It is possible to shrink the component dimension and operate at a higher rate for same power density. This is a result of smaller capacitance associated with smaller device area. Consider a given power density

⁷Source: <https://www.kaggle.com/datasets/michaelbryantds/cpu-and-gpu-product-data>.

⁸GPUs are special-purpose chips intended for graphics rendering. However, due to their parallel processing capability, they are also used for applications like Artificial Intelligence and Machine Learning.

⁹TDP is the maximum amount of heat the chip can generate which the cooling system is capable of dissipating at the base frequency

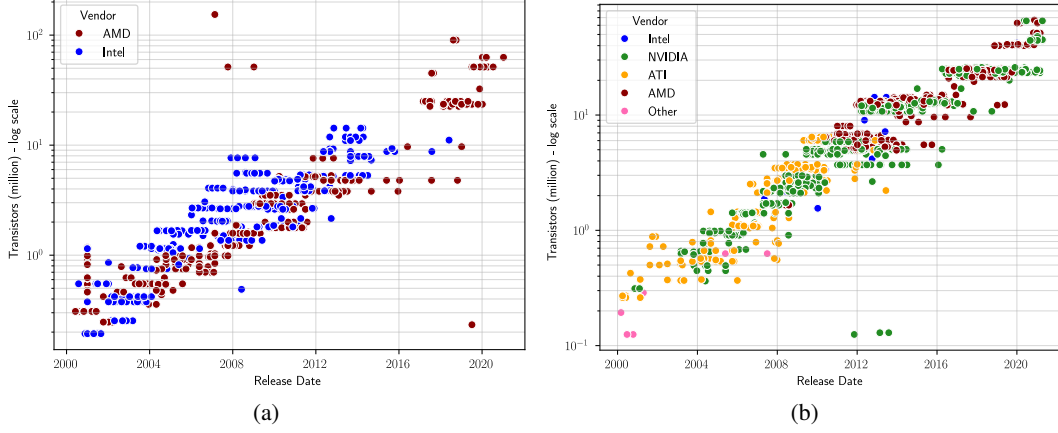


Figure 3: Transistor count (in millions) per unit mm² in (a) CPUs and (b) GPUs plotted against year of release.

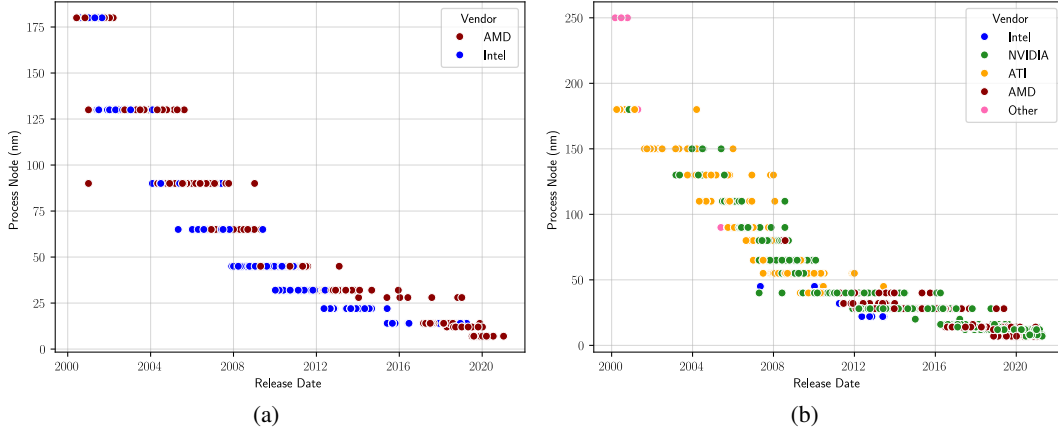


Figure 4: Process Node (in nm) technology used in production of (a) CPUs and (b) GPUs plotted against year of release.

P (power units per unit area). The power consumed per unit area can be roughly estimated by:

$$P = \gamma \cdot C \cdot V^2 \cdot f, \quad (1)$$

where C is capacitance per unit area, V is operating voltage, f is clock frequency, and γ is a proportionality constant.

Thus, it is possible to increase f with smaller C and maintain the power density.

The reduction in power consumption as seen in Figure 5 means today we have CPU chips rated at 120W TDP¹⁰. The plots show a consistent decline in the power dissipation (in μ W) per transistor until the late 2000s. Post 2008, the reduction in power consumption has relatively decreased.

3.3 Realization of Moore's Law

Moore's law forecasted the growth of the integrated electronics through improvements in the fabrication and manufacturing techniques. Although, it did not suggest explicit ways to do so various methods were engineered at different time periods.

The initial phase, often referred to as Moore's Law 1.0, relied heavily on reducing transistor dimensions to increase the number of components per chip. This trend enabled consistent improvements in performance and energy efficiency.

¹⁰Source: <https://www.tomshardware.com/reviews/cpu-hierarchy,4312.html>

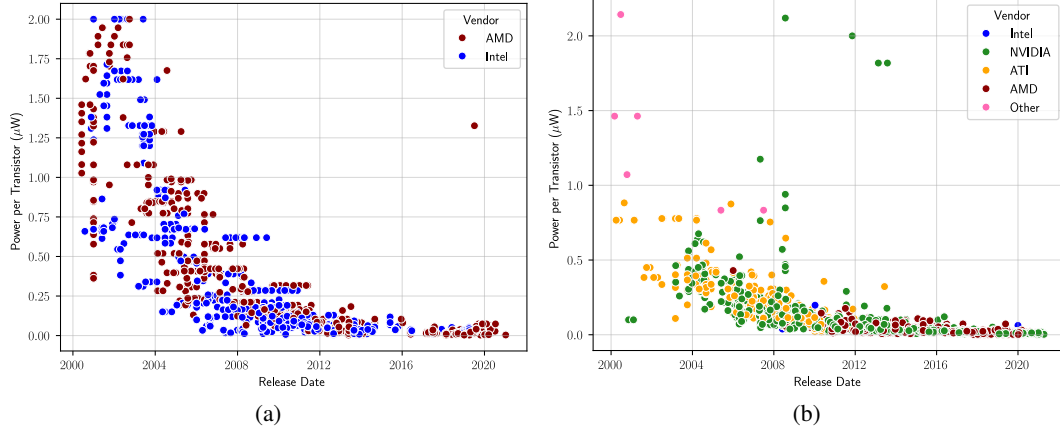


Figure 5: Power density in terms of μW per transistor in (a) CPUs and (b) GPUs plotted against year of release.

Subsequently, Moore's Law 2.0 focused on increasing the clock frequency of processors to boost computational throughput. However, this approach faced limitations due to the thermal challenges posed by higher frequencies. Efficient heat dissipation became a bottleneck, particularly in CPUs, as illustrated in Figure 6. Interestingly, GPUs, which historically operate at lower clock speeds than CPUs, continue to show incremental gains in frequency.

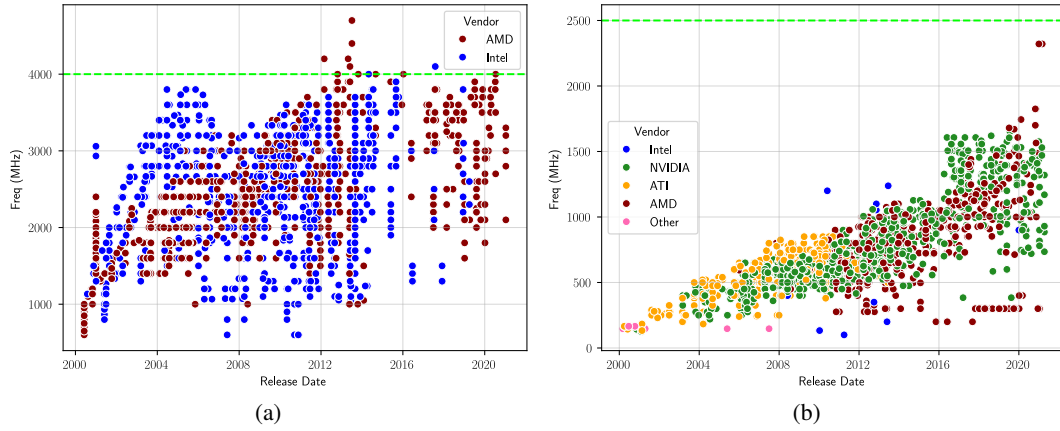


Figure 6: Clock frequency (in MHz) of (a) CPUs and (b) GPUs plotted against year of release.

4 Conclusion — The Past, Present, and Future

Following the end of Dennard scaling and the stagnation of frequency scaling, Moore's Law 3.0 emerged. This phase introduced enhanced functionality by integrating additional components on a single chip. Beginning with multi-core (additional CPU cores on a single chip) architectures, the trend evolved into fully integrated platforms including graphics processors, wireless radios, memory controllers, and high-speed interconnects—collectively known as System-on-Chip (SoC) designs.

These Moore's Law evolutions are often categorized into broader frameworks:

More Moore: Continuation of traditional CMOS¹¹ scaling, including techniques like constant-field or constant-voltage scaling (i.e., Dennard scaling).

More than Moore: Integration of non-digital or heterogeneous functions that enhance application-specific capabilities without relying solely on transistor scaling.

¹¹CMOS stands for Complementary MOS. It is the predominant technology used today.

Beyond Moore: Exploration of fundamentally different computing paradigms. This includes three-dimensional integration, use of novel materials (e.g., carbon nanotube, Graphene), quantum computing principles, and biologically inspired architectures. These approaches primarily explore the spatial domain of performance enhancement.

Another dimension, termed Time Moore [Xiu \(2019\)](#), leverages temporal strategies—such as clock optimization (Clock Moore) and adaptive data rates (Rate Moore)—to continue improving performance.

Gordon Moore’s original 1965 prediction suggested that chip complexity would reach approximately 65,000 transistors by 1975 and that growth-rate would continue till at least until late 2000s. Modern integrated circuits today exceed one billion transistors, with some specialized chips reaching 208 billion¹². Moore’s insight continues to shape the trajectory of semiconductor technology decades later.

¹²Source: https://en.wikipedia.org/wiki/Transistor_count

References

- Moore, G. (1998). Cramming more components onto integrated circuits. *Proceedings of the IEEE*, 86(1):82–85.
- Xiu, L. (2019). Time moore: Exploiting moore’s law from the perspective of time. *IEEE Solid-State Circuits Magazine*, 11(1):39–55.